



Module 10 – Kubernetes Security

EKS Anywhere Training Program · Advanced Security for Production Clusters

ENTERPRISE SECURITY

EKS ANYWHERE

OVERVIEW

Kubernetes Security Overview

Securing a Kubernetes cluster requires a layered, defense-in-depth approach. No single control is sufficient — production-grade security spans identity, access, runtime, and supply chain. This module covers the four critical security layers enforced at the API Server boundary and beyond.

Authentication

Verify who is making the request — users, services, and external systems

Authorization

Determine what actions the authenticated identity is permitted to take

Admission Control

Enforce policy on resource creation and mutation before persistence

Data Security

Protect secrets at rest, etcd encryption, and TLS in transit

Why Kubernetes Security Matters

Kubernetes' default configuration is optimized for functionality, not security. Misconfigured clusters are among the top vectors for cloud-native breaches — from exposed API servers to over-privileged service accounts.

Blast Radius

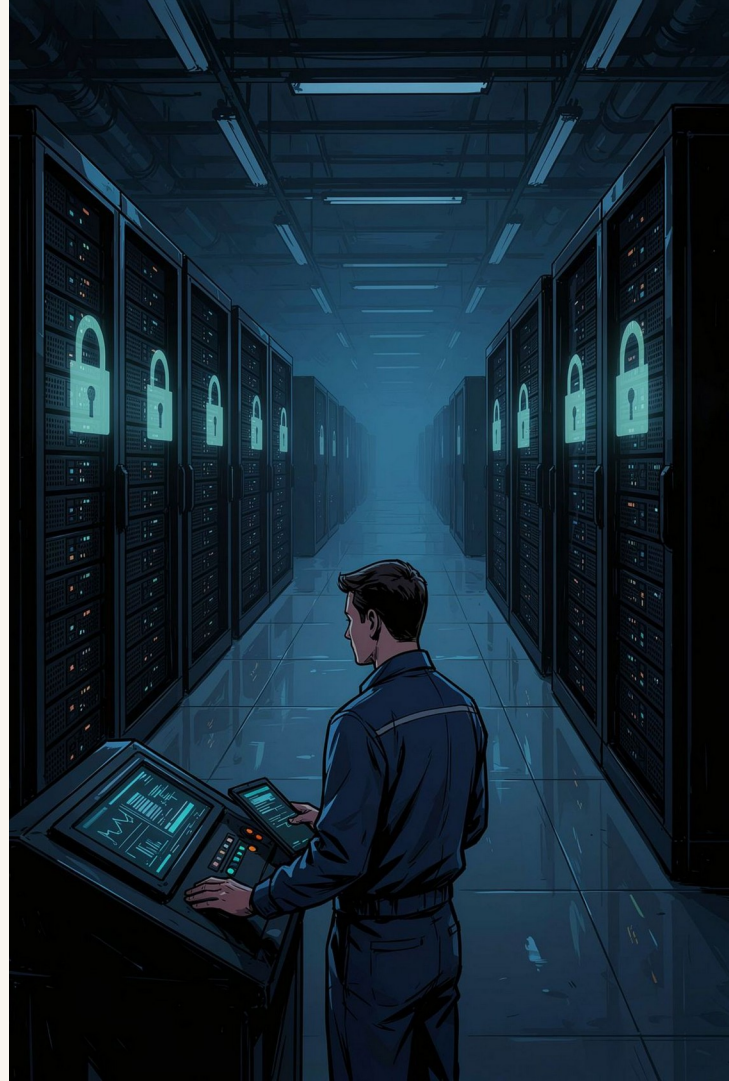
A single compromised pod with excessive RBAC permissions can pivot to full cluster takeover. Least-privilege is non-negotiable.

Supply Chain Risk

Untrusted container images and unsigned artifacts introduce runtime vulnerabilities invisible to RBAC controls.

Compliance Mandates

PCI-DSS, SOC 2, HIPAA, and FedRAMP all impose specific controls on container workloads that must be enforced via policy.



Kubernetes Security Architecture

Every request to the API Server traverses a strict security pipeline. Understanding this flow is foundational to designing effective controls — each layer is a gate that must be passed before a resource is persisted to etcd.



Requests that fail any stage are rejected with an appropriate HTTP error code. Only fully validated and admitted requests are written to the etcd datastore, which should itself be encrypted at rest using AES-GCM or AES-CBC providers.

Authentication Methods

Kubernetes supports multiple authentication strategies simultaneously. The API Server tries each configured authenticator in order — the first successful match wins. In EKS Anywhere environments, IAM and OIDC are the preferred enterprise patterns.



X.509 Certificates

Mutual TLS for kubelets, control-plane components, and kubectl. Managed via the cluster CA. Short-lived certs recommended.



Service Accounts

Pod-scoped JWT tokens projected by the kubelet. Bound to namespace and audience. Expiry configurable (default 1h in modern clusters).



OIDC / JWT

Federate corporate IdPs (Okta, Entra ID, Dex) for human operator access. Token expiry enforced. Preferred for SSO environments.



Webhook Token Auth

Custom external authentication via HTTP webhook. Enables integration with HashiCorp Vault, custom IAM systems, or legacy PKI.



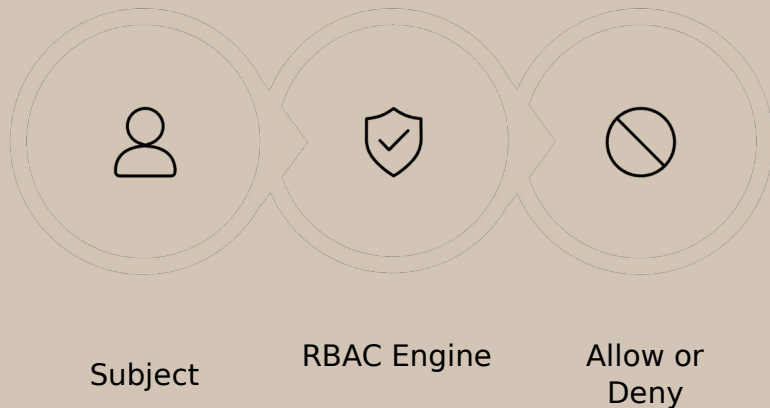
AWS IAM (EKS)

IAM Authenticator maps IAM roles/users to Kubernetes RBAC via aws-auth ConfigMap or EKS Access Entries API.

Authorization Flow and RBAC

Once authenticated, every request is evaluated by the authorization module. Kubernetes supports Node, ABAC, RBAC, and Webhook authorizers — but **RBAC is the only recommended mode** for production clusters. ABAC should be explicitly disabled.

RBAC Decision Flow



Key Properties

- **Default-deny**: No permissions unless explicitly granted
- **Additive only**: RBAC cannot deny — use admission or network policy for explicit blocks
- **Namespace-scoped**: Roles apply within a namespace; ClusterRoles apply cluster-wide
- **Aggregation**: ClusterRoles can aggregate from multiple labeled roles
- **Audit**: All authorization decisions are recorded in the Kubernetes audit log

RBAC Fundamentals

RBAC is built on four core API objects. Understanding the binding between subjects and roles — and the scope of each object — is essential before designing any production access control model.

1

Role

Namespaced. Defines allowed verbs on specific resources within a single namespace. Cannot grant cross-namespace access.

2

ClusterRole

Cluster-scoped. Can grant permissions on namespaced or non-namespaced resources (nodes, PVs, CRDs). Also used for cross-namespace bindings.

3

RoleBinding

Namespaced. Binds a Role or ClusterRole to subjects within a specific namespace. Grants scoped access only.

4

ClusterRoleBinding

Cluster-scoped. Binds a ClusterRole to subjects across all namespaces. Use sparingly — high blast radius if over-privileged.

📌 Production rule: Prefer **RoleBinding + ClusterRole** over ClusterRoleBinding to scope cluster-wide permissions per namespace.

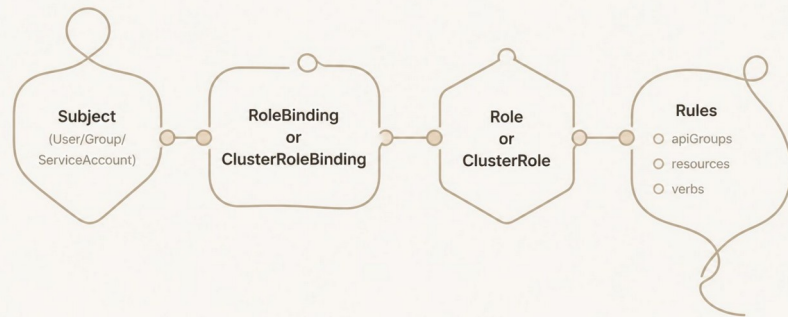
RBAC Permission Model

RBAC permissions are defined along three axes: API group, resource type, and allowed verbs. Understanding these dimensions enables precise least-privilege configurations and effective auditing of access.

Permission Dimensions

Axis	Examples	Notes
apiGroups	<code>"", apps, batch</code>	Core group is empty string
resources	pods, deployments, secrets	Use subresources: pods/log
verbs	get, list, watch, create, patch, delete	Avoid wildcard <code>*</code>
subjects	User, Group, ServiceAccount	SA must specify namespace

Binding Flow



RBAC YAML Reference

The following patterns represent production-ready RBAC configurations. Note the separation of role definition from binding — this enables role reuse across multiple subjects and namespaces.

Role + RoleBinding (Namespaced)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: production
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
---
kind: RoleBinding
metadata:
  name: read-pods
  namespace: production
subjects:
- kind: ServiceAccount
  name: app-monitor
  namespace: production
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Key Field Reference

- **apiGroups: [""]** — core API group (pods, services, secrets)
- **apiGroups: ["apps"]** — deployments, replicaset, daemonsets
- **verbs** — always enumerate explicitly; never use "*" in production
- **subjects[].namespace** — required for ServiceAccount subjects
- **roleRef** — immutable after binding creation; delete and recreate to change



Never grant **secrets: ["get", "list"]** broadly — this exposes all secrets in the namespace to the bound subject.

RBAC Best Practices & Least Privilege

Effective RBAC design is an ongoing operational discipline, not a one-time configuration. Drift, over-provisioning, and stale bindings are the most common production failure modes.

→ Audit Regularly

Use `kubectl auth can-i --list --as=system:serviceaccount:ns:name` and tools like **rbac-tool** or **Kubescape** to enumerate effective permissions continuously.

→ Minimize ClusterRoleBindings

Prefer RoleBinding + ClusterRole for namespace-scoped access. Reserve ClusterRoleBindings for genuine cluster-admin use cases and audit them weekly.

→ Avoid Wildcards

Never use `resources: ["*"]` or `verbs: ["*"]` in production roles. Be explicit about every permitted action to minimize blast radius.

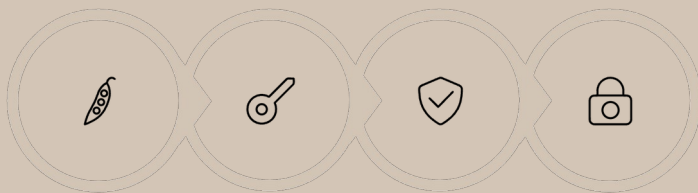
→ Separate Human vs. Workload Identities

Never reuse a ServiceAccount for human operators. Human access should use OIDC or certificate-based identities with short TTLs and MFA where possible.

Service Account Architecture

Service Accounts are the primary identity mechanism for workloads running inside the cluster. Modern Kubernetes uses **projected service account tokens** — audience-bound, time-limited JWTs injected by the kubelet rather than long-lived static secrets.

Token Flow



Pod requests
API

Kubelet
injects token

API Server
validates

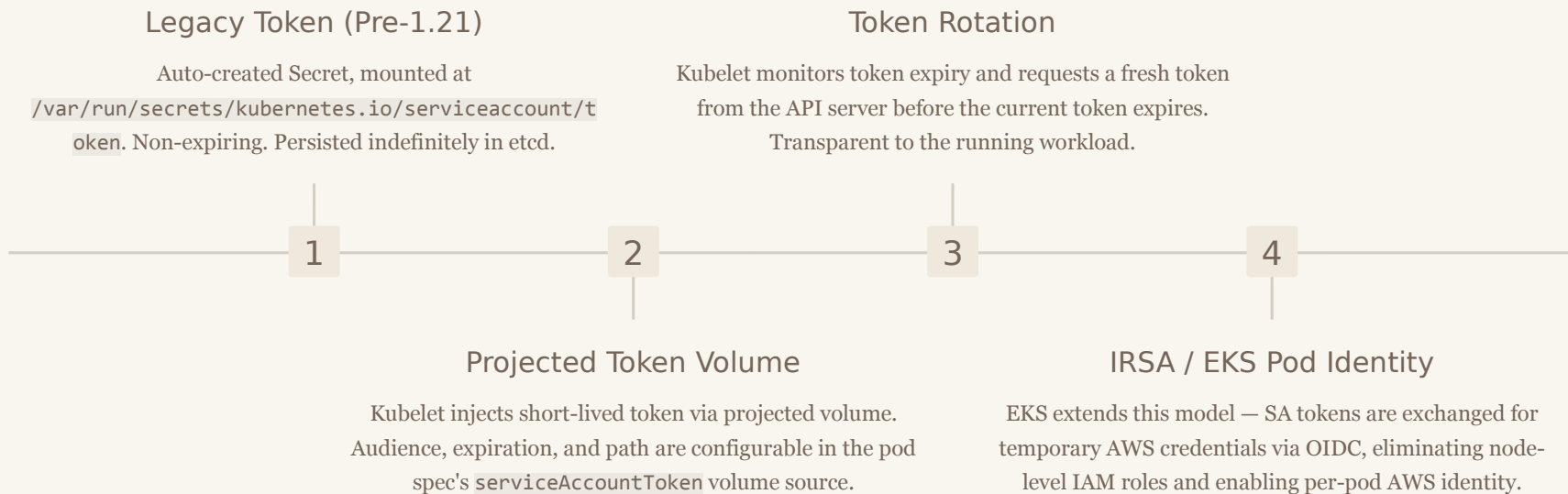
RBAC
evaluates SA

Architecture Properties

- **Audience binding** — tokens are scoped to a specific audience (e.g., the API server or external OIDC consumer)
- **Expiry enforcement** — default 1-hour TTL; kubelet auto-rotates before expiry
- **Namespace-scoped** — a ServiceAccount exists only within its namespace
- **automountServiceAccountToken: false** — disable auto-mount on pods that don't require API access
- **IRSA / Pod Identity** — on EKS, OIDC federation maps SA to IAM role for AWS API access

Service Account Lifecycle & Projected Tokens

The projected service account token volume type replaced the legacy `kubernetes.io/service-account-token` Secret. This shift eliminates long-lived, non-rotatable secrets from pods and is the default behavior since Kubernetes 1.21.



📌 Migrate all workloads off legacy token Secrets. Audit with: `kubectl get secrets --all-namespaces | grep service-account-token`

Service Account Production Best Practices

Service Account security is one of the highest-leverage controls in a Kubernetes cluster. Default behaviors are insecure — every production deployment should explicitly configure SA settings.



Disable Auto-Mount

Set `automountServiceAccountToken: false` on both the ServiceAccount object and individual pods that don't call the Kubernetes API.



One SA per Workload

Never share ServiceAccounts across different applications. Unique SAs enable precise RBAC scoping and clean audit trails per workload identity.



Least-Privilege RBAC

Bind only the minimum permissions required. Periodically review with `kubectl auth can-i --list` and automated policy tools like Kyverno.

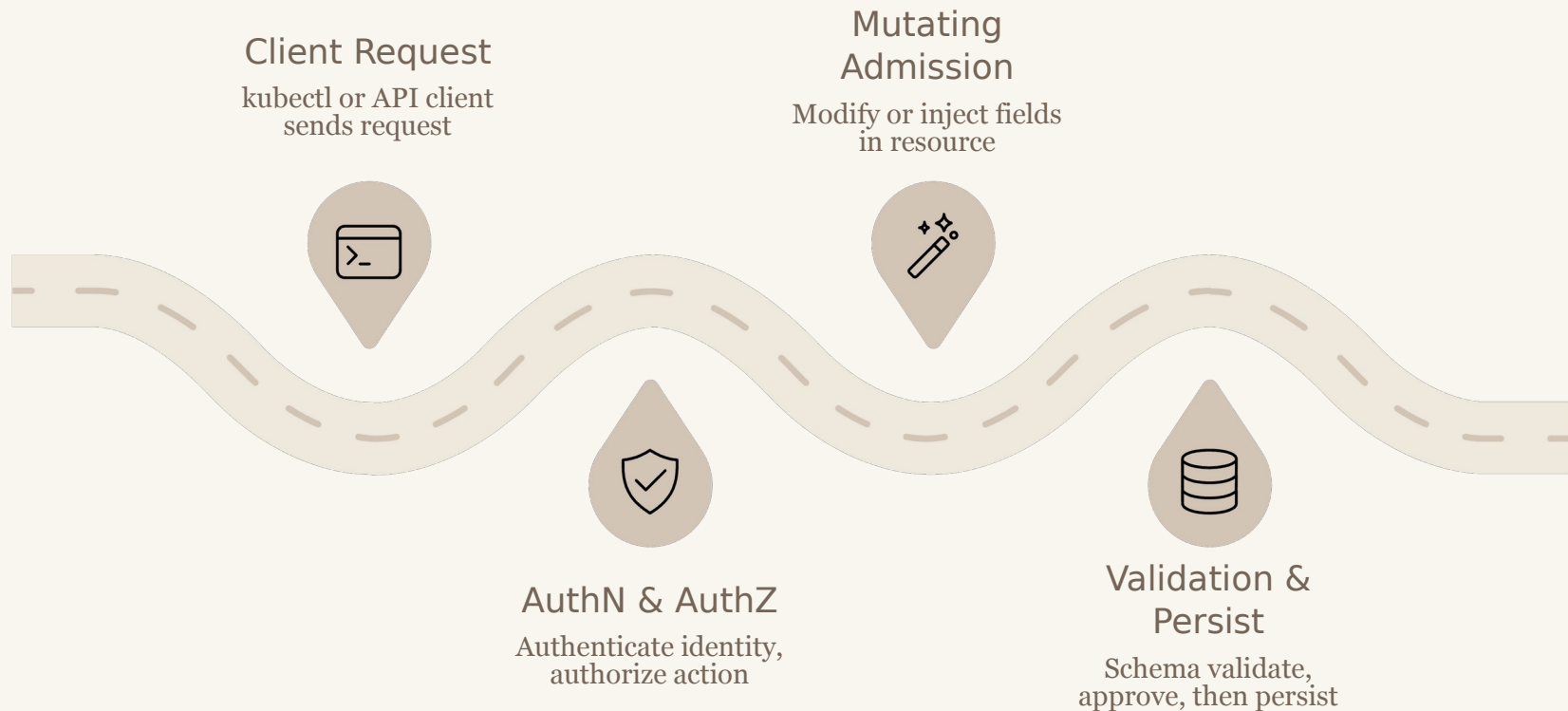


Use IRSA / Pod Identity

For AWS API access, always use EKS IRSA or EKS Pod Identity. Never attach IAM policies to node instance profiles for workload-specific access.

Admission Controllers Architecture

Admission controllers are API Server plugins that intercept requests **after authentication and authorization**, but before persistence to etcd. They represent the last enforcement boundary for policy within the Kubernetes control plane.



The pipeline executes all **Mutating** webhooks first (in parallel), then re-validates the object schema, then runs all **Validating** webhooks (also in parallel). A single rejection from any validating controller blocks the request.

Mutating vs. Validating Admission Controllers

Understanding the distinction between mutation and validation is critical for designing reliable admission policy. Mutating webhooks modify requests silently — this can introduce unexpected state if not carefully controlled.

Dimension	Mutating	Validating
Purpose	Modify the resource object	Approve or reject the request
Execution Order	First (before validation)	Second (after mutation)
Can Modify Object	Yes (JSON Patch)	No
Example Use Cases	Inject sidecars, set defaults, add labels/annotations	Enforce required labels, block privileged pods, require image digests
Failure Mode	failurePolicy: Fail or Ignore	failurePolicy: Fail or Ignore
Real-world Tools	Istio, Linkerd, Kyverno mutate rules	OPA Gatekeeper, Kyverno validate, PSA

⚠ Set `failurePolicy: Fail` for security-critical webhooks. `Ignore` silently allows requests if the webhook is unavailable — a dangerous default in production.

Production Admission Controllers

A hardened EKS Anywhere cluster should enable and configure the following admission controllers as a baseline security posture. These controllers enforce resource governance, pod security, and policy compliance at admission time.

Pod Security Admission

Enforces **privileged**, **baseline**, or **restricted** Pod Security Standards per namespace. Replaces deprecated PodSecurityPolicy. Label namespaces with `pod-security.kubernetes.io/enforce: restricted`.

LimitRanger & ResourceQuota

LimitRanger sets default CPU/memory requests and limits per container. ResourceQuota enforces aggregate namespace-level consumption caps. Together they prevent noisy-neighbor resource exhaustion.

ValidatingAdmissionPolicy (CEL)

Native Kubernetes policy engine introduced in 1.26 (GA in 1.30). Write policy as Common Expression Language (CEL) directly in the API — no external webhook required. Preferred for greenfield deployments.

MutatingAdmissionWebhook

Dynamic webhook for sidecar injection (Istio, Linkerd), label stamping, and default field injection. Register with precise `namespaceSelector` and `objectSelector` to minimize scope.

ValidatingAdmissionWebhook

Dynamic webhook for external policy engines: OPA Gatekeeper, Kyverno, jsPolicy. Provides the richest policy expressiveness and ecosystem integrations for complex enterprise compliance requirements.

Enterprise Kubernetes Security Best Practices

A production-hardened cluster implements defense-in-depth across every layer. The following practices represent the enterprise security baseline for EKS Anywhere deployments.

RBAC & Least Privilege

Audit bindings weekly. No wildcards. Prefer RoleBinding over ClusterRoleBinding.

Service Account Hygiene

One SA per workload. Disable auto-mount. Use IRSA/Pod Identity for AWS access.

Pod Security Standards

Enforce restricted profile on all namespaces. Baseline for exceptions with justification.

Network Policies

Default-deny ingress and egress per namespace. Allow only required traffic explicitly.

Secrets Encryption

Enable etcd encryption at rest with AES-GCM or KMS provider. Rotate encryption keys quarterly.

Image Signing

Sign all images with Cosign/Sigstore. Enforce signature verification via Kyverno or Connaisseur at admission.

Audit Logging

Enable API server audit policy. Ship logs to immutable SIEM. Alert on sensitive verb patterns.

Continuous Compliance

Scan clusters with Kubescape, Trivy Operator, or AWS Security Hub. Enforce policy drift via GitOps.

Kyverno & OPA Gatekeeper

For production clusters requiring rich policy expressiveness beyond built-in admission controllers, Kyverno and OPA Gatekeeper are the two dominant open-source policy engines. Both integrate via ValidatingAdmissionWebhook.

Kyverno

- **Kubernetes-native YAML** — policies are Kubernetes CRDs, no new language required
- Supports **validate, mutate, generate, and verify image** rules in a single policy
- Built-in **image signature verification** via Cosign integration
- Background scanning of existing resources for drift detection
- Best for: teams preferring YAML-native policy without learning OPA/Rego

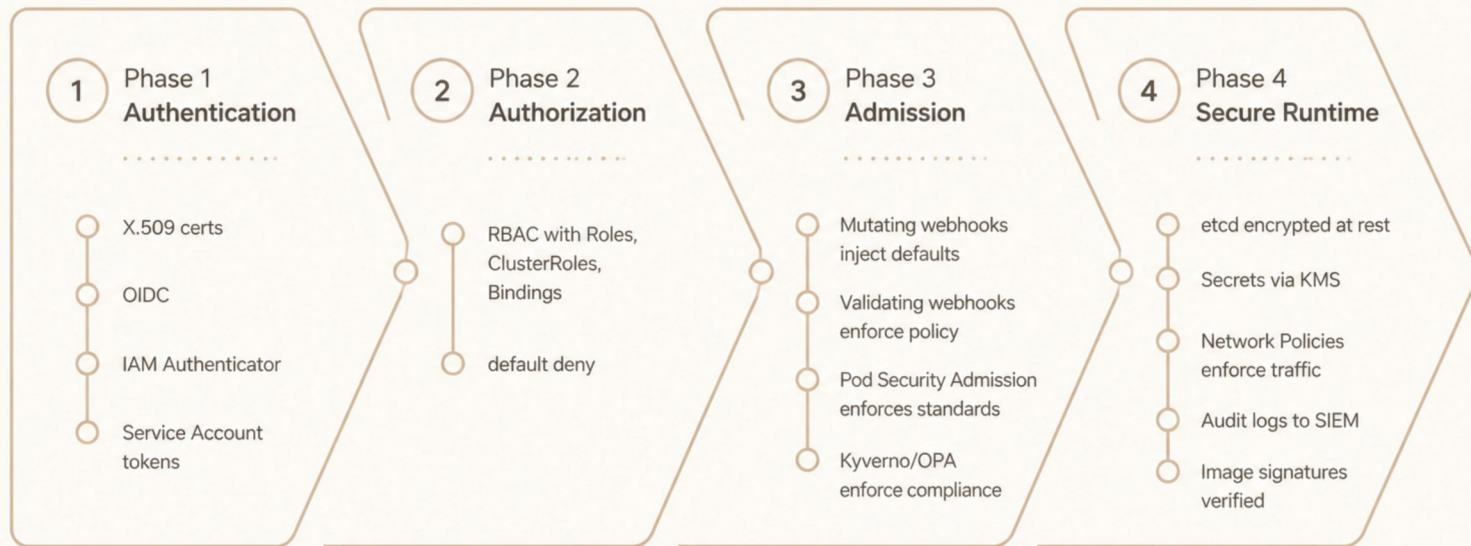
OPA Gatekeeper

- **Rego language** — highly expressive, context-aware policy evaluation
- ConstraintTemplate + Constraint pattern enables policy parameterization
- Supports **audit mode** to report violations without blocking
- Integrates with OPA ecosystem (Conftest, policy testing frameworks)
- Best for: teams with existing Rego expertise or complex policy requirements

 Both tools support **dry-run / audit mode** — use this to assess policy impact before enforcing in production namespaces.

End-to-End Secure Cluster Workflow

Every production Kubernetes request passes through a complete security pipeline. Gaps at any layer create exploitable attack surfaces. The following workflow represents the integrated security model for an EKS Anywhere production cluster.



Module 10 Complete

You now have a production-grade mental model for Kubernetes security across all four enforcement layers.

01

Authentication

X.509, OIDC, IAM Authenticator, Projected Service Account Tokens

02

Authorization

RBAC with least-privilege Roles, Bindings, and continuous audit

03

Admission Control

Mutating + Validating webhooks, PSA, CEL policies, Kyverno/Gatekeeper

04

Runtime & Compliance

etcd encryption, image signing, audit logging, continuous policy scanning

❏ Next: **Module 11 - EKS Anywhere Networking**. Lab exercises for this module are available in the training portal.

